

Язык программирования C++

Лекция 12: наследование, полиморфизм, виртуальные функции

Никита Лисица

2026

Напоминание про наследование

- Позволяет нам реализовать один класс поверх другого, переиспользуя его поля и методы:

```
class Button {  
protected:  
    const char* caption_;  
    Action clickAction_;  
  
public:  
    ...  
  
    virtual void draw() {  
        drawRectangle(Color::Gray);  
        drawText(caption_);  
    }  
  
    virtual void click() {  
        clickAction_.execute();  
    }  
};
```

Напоминание про наследование

- Позволяет нам реализовать один класс поверх другого, переиспользуя его поля и методы:

```
class ColoredButton : public Button {  
protected:  
    Color color_;  
  
public:  
    ...  
  
    // Автоматически virtual  
    void draw() override {  
        drawRectangle(color_);  
        drawText(caption_);  
    }  
  
    // Автоматически virtual  
    void click() override {  
        color_ = Color::Red;  
        Button::click();  
    }  
};
```

Override

- Допустим, мы перепутали название метода в классе-наследнике:

```
class ColoredButton : public Button {  
public:  
    void drawButton() { ... }  
    ...  
};  
  
Button* button = new ColoredButton(...);  
button->draw(); // Что произойдёт?
```

Override

- Допустим, мы перепутали название метода в классе-наследнике:

```
class ColoredButton : public Button {  
public:  
    void drawButton() { ... }  
    ...  
};  
  
Button* button = new ColoredButton(...);  
button->draw(); // Что произойдёт?
```

- Вызовется метод `Button::draw()`, так как метод `ColoredButton::drawButton()` не имеет к нему никакого отношения

Override

- Ключевое слово **override** заставит компилятор проверить, что метод *переопределяет* какой-то виртуальный метод базового класса

Override

- Ключевое слово **override** заставит компилятор проверить, что метод *переопределяет* какой-то виртуальный метод базового класса

```
class ColoredButton : public Button {  
public:  
    // Ошибка компиляции  
    void drawButton() override { ... }  
    ...  
};
```

Override

- Ключевое слово **override** заставит компилятор проверить, что метод *переопределяет* какой-то виртуальный метод базового класса

```
class ColoredButton : public Button {  
public:  
    // Ошибка компиляции  
    void drawButton() override { ... }  
    ...  
};
```

- Хорошая идея – **всегда** писать **override**, когда вы переопределяете виртуальный метод

Override vs overload

- Переопределение методов (override) не стоит путать с перегрузкой (overload) функций или методов

Override vs overload

- Переопределение методов (override) не стоит путать с перегрузкой (overload) функций или методов
- Переопределение: метод в классе-наследнике перекрывает виртуальный метод базового класса:

```
class B : public A {  
public:  
    void exec() override { ... }  
};
```

Override vs overload

- Переопределение методов (override) не стоит путать с перегрузкой (overload) функций или методов
- Переопределение: метод в классе-наследнике перекрывает виртуальный метод базового класса:

```
class B : public A {  
public:  
    void exec() override { ... }  
};
```

- Перегрузка: две функции (или два метода одного класса) имеют одинаковое имя, но разные аргументы:

```
class B {  
public:  
    void exec(int x) { ... }  
    void exec(float x) { ... }  
};
```

Пример: странный связный список, list.h

```
class List {  
private:  
    int value;  
    List* next;  
  
public:  
    List(int value);  
    ~List();  
  
    void push_back(int value);  
    size_t length() const;  
};
```

Пример: странный связный список, list.cpp

```
#include "list.h"

List::List(int value) {
    this->value = value; this->next = nullptr;
}

~List::List() { /* Упражнение :) */ }

void List::push_back(int value) {
    List* current = this;
    while (current->next != NULL)
        current = current->next;
    current->next = new List(value);
}

size_t List::length() const {
    // В константном методе this имеет тип const List* const
    size_t count = 0; const List* current = this;
    while (current->next != NULL) {
        current = current->next; ++count;
    }
    return count;
}
```

Пример: странный связный список

- Почему он странный?

Пример: странный связный список

- Почему он странный?
- Смешаны понятия списка и одного узла (node)

Пример: странный связный список

- Почему он странный?
- Смешаны понятия списка и одного узла (node)
- Не бывает пустым

Пример: странный связный список

- Почему он странный?
- Смешаны понятия списка и одного узла (node)
- Не бывает пустым
- Все операции за линейное время

Пример: странный связный список

- Почему он странный?
- Смешаны понятия списка и одного узла (node)
- Не бывает пустым
- Все операции за линейное время
- Это не страшно, пример педагогический :)

Пример: странный двусвязный список, `double_list.h`

- Позже решили добавить двусвязный список, отнаследовшись от уже имеющегося – многие операции вообще не изменятся: `push_back()`, `length()`

Пример: странный двусвязный список, double_list.h

- Позже решили добавить двусвязный список, отнаследовшись от уже имеющегося – многие операции вообще не изменятся: `push_back()`, `length()`

```
#include "list.h"

class DoubleList : public List {
private:
    DoubleList* prev;

public:
    DoubleList(int val);
    ~DoubleList();
    void push_back(int val);
    void pop_back(); // Новый метод!
};
```

Пример: странный двусвязный список, double_list.cpp

```
#include "double_list.h"

DoubleList::DoubleList(int value)
    : List(value), prev(nullptr)
{}

~DoubleList::DoubleList() { /* Упражнение :) */ }

void DoubleList::push_back(int value) {
    DoubleList* current = this;
    while (current->next != NULL)
        current = current->next;
    current->next = new DoubleList(value);
    current->next->prev = current;
}

void DoubleList::pop_back() { /* Упражнение :) */ }
```

Пример: странный двусвязный список

- В примере есть три ошибки:

Пример: странный двусвязный список

- В примере есть три ошибки:
 1. Поля `class List` должны быть не `private`, а `protected`

Пример: странный двусвязный список

- В примере есть три ошибки:
 1. Поля `class List` должны быть не `private`, а `protected`
 2. У метода `push_back()` не хватает `virtual` и `override`

Пример: странный двусвязный список

- В примере есть три ошибки:
 1. Поля `class List` должны быть не `private`, а `protected`
 2. У метода `push_back()` не хватает `virtual` и `override`

```
class List {  
protected:  
    int val;  
    List *next;  
  
public:  
    virtual void push_back(int val);  
};  
  
class DoubleList : public List {  
    ...  
public:  
    void push_back(int val) override;  
};
```

Пример: странный двусвязный список

- В примере есть три ошибки:

Пример: странный двусвязный список

- В примере есть три ошибки:
 3. В методе `DoubleList::push_back()` некорректное приведение типов `List* -> DoubleList*`

Пример: странный двусвязный список

- В примере есть три ошибки:
 3. В методе `DoubleList::push_back()` некорректное приведение типов `List* -> DoubleList*`

```
// Ошибка: приведение List* к DoubleList*
current = current->next;

// ОК: приведение DoubleList* к List*
current->next = new DoubleList(value);

// Ошибка: current->next имеет тип List*, у него
// нет поля prev
current->next->prev = current;
```

Приведение типов при наследовании

- При публичном наследовании класс-наследник ‘умеет’ всё, что умеет базовый класс, содержит тот же публичный интерфейс (поля и методы)

Приведение типов при наследовании

- При публичном наследовании класс-наследник 'умеет' всё, что умеет базовый класс, содержит тот же публичный интерфейс (поля и методы) $\implies$ указатель на класс-наследник неявно приводится к указателю на базовый класс

Приведение типов при наследовании

- При публичном наследовании класс-наследник ‘умеет’ всё, что умеет базовый класс, содержит тот же публичный интерфейс (поля и методы) $\implies$ указатель на класс-наследник неявно приводится к указателю на базовый класс
- Обратное, вообще говоря, неверно (например, у `List` нет поля `prev` и метода `pop_back`)

Приведение типов при наследовании

- При публичном наследовании класс-наследник ‘умеет’ всё, что умеет базовый класс, содержит тот же публичный интерфейс (поля и методы) $\implies$ указатель на класс-наследник неявно приводится к указателю на базовый класс
- Обратное, вообще говоря, неверно (например, у `List` нет поля `prev` и метода `pop_back`)
- Если мы точно знаем, что по данному указателю лежит конкретный класс-наследник, можно сделать *явное приведение типов*:

```
List* list = ...;

// Знаем, что там на самом деле DoubleList:
DoubleList* dlist = (DoubleList*)list;
dlist->pop_back();
```

Исправление ошибки

```
void DoubleList::push_back(int value) {  
    DoubleList* current = this;  
    while (current->next != NULL)  
        current = (DoubleList*)current->next;  
    current->next = new DoubleList(value);  
    ((DoubleList*)current->next)->prev = current;  
}
```

Наследование: плюсы

- Получается гибкий, расширяемый код

```
// Функцию написали задолго до
// появления DoubleList
void fill(List* l, int val, size_t num) {
    for(size_t i = 0; i < num; i++)
        l->push_back(val);
}

// Код работает без изменений в
// функции fill
List l(5);
DoubleList dl(6);

fill(&l, 10, 42);
fill(&dl, 100, 24);
```

Пример: логирование

- Задача: хотим реализовать расширяемую систему логирования

Пример: логирование

- Задача: хотим реализовать расширяемую систему логирования
- Должна позволять логировать сообщения одновременно в разные места (т.н. *sinks* – в `stdout`, в файл, и т.п.)

Пример: логирование

- Задача: хотим реализовать расширяемую систему логирования
- Должна позволять логировать сообщения одновременно в разные места (т.н. *sinks* – в `stdout`, в файл, и т.п.)
- Должна поддерживать уровни логирования:
ERROR, WARNING, INFO, DEBUG, TRACE

Пример: логирование

- Задача: хотим реализовать расширяемую систему логирования
- Должна позволять логировать сообщения одновременно в разные места (т.н. *sinks* – в `stdout`, в файл, и т.п.)
- Должна поддерживать уровни логирования:
`ERROR`, `WARNING`, `INFO`, `DEBUG`, `TRACE`
- При добавлении новых `sink`'ов не хочется переписывать весь код для их поддержки

Пример: логирование

- Хотим, чтобы код выглядел как-то так:

```
Logger logger;  
  
StdoutSink stdoutSink(WARNING);  
FileSink fileSink1("log.txt", WARNING);  
FileSink fileSink2("debug_log.txt", DEBUG);  
logger.addSink(stdoutSink);  
logger.addSink(fileSink1);  
logger.addSink(fileSink2);  
  
logger.log(INFO, "Initialized");  
logger.log(DEBUG, "Start time: ...");
```

Пример: логирование

- Хотим, чтобы код выглядел как-то так:

```
Logger logger;  
  
StdoutSink stdoutSink(WARNING);  
FileSink fileSink1("log.txt", WARNING);  
FileSink fileSink2("debug_log.txt", DEBUG);  
logger.addSink(stdoutSink);  
logger.addSink(fileSink1);  
logger.addSink(fileSink2);  
  
logger.log(INFO, "Initialized");  
logger.log(DEBUG, "Start time: ...");
```

- Хотим сделать так, чтобы при добавлении новых sink'ов класс **Logger** не пришлось переделывать

Пример: логирование

- Базовый класс:

```
class Sink {  
protected:  
    Level level_;  
public:  
    Sink(Level level): level_(level) {}  
    virtual ~Sink() {}  
    virtual void log(Level level, const char* message) {}  
};
```

Пример: логирование

- Реализация логирования в `stdout`:

```
class StdoutSink : public Sink {
public:
    StdoutSink(Level level): Sink(level_) {}

    void log(Level level, const char* message) override {
        if (level >= level_)
            printf(message);
    }
};
```

Пример: логирование

- Реализация логирования в файл:

```
class FileSink : public Sink {
protected:
    FILE* file_;
public:
    FileSink(const char* filename, Level level)
        : Sink(level_)
        , file_(fopen(filename, "at"))
    {}

    void log(Level level, const char* message) override {
        if (level >= level_)
            fprintf(file_, message);
    }

    ~FileSink() override {
        fclose(file_);
    }
};
```

Пример: логирование

- Реализация основного класса `Logger`:

```
class Logger {  
private:  
    Sink sinks_[100];  
    size_t size_;  
public:  
    Logger(): size_(0) {}  
  
    void addSink(Sink& sink) {  
        sinks_[size_++] = sink;  
    }  
  
    void log(Level level, const char* message) {  
        for (size_t i = 0; i < size_; ++i)  
            sinks_[i].log(level, message);  
    }  
};
```

Пример: логирование

- Реализация основного класса `Logger`:

```
class Logger {  
private:  
    Sink sinks_[100];  
    size_t size_;  
public:  
    Logger(): size_(0) {}  
  
    void addSink(Sink& sink) {  
        sinks_[size_++] = sink;  
    }  
  
    void log(Level level, const char* message) {  
        for (size_t i = 0; i < size_; ++i)  
            sinks_[i].log(level, message);  
    }  
};
```

- Будет ли работать?

Пример: логирование

- Реализация основного класса `Logger`:

```
class Logger {  
private:  
    Sink sinks_[100];  
    size_t size_;  
public:  
    Logger(): size_(0) {}  
  
    void addSink(Sink& sink) {  
        sinks_[size_++] = sink;  
    }  
  
    void log(Level level, const char* message) {  
        for (size_t i = 0; i < size_; ++i)  
            sinks_[i].log(level, message);  
    }  
};
```

- Будет ли работать? Нет, так как `Logger` хранит *копии* переданных `sink`'ов, к тому же в виде объектов класса `Sink`

- При публичном наследовании объект класса-наследника неявно приводится к типу базового класса

Пример: логирование

- При публичном наследовании объект класса-наследника неявно приводится к типу базового класса
- $\Rightarrow$ Такой код скопируется, хотя смысла в нём немного:

```
FileSink fileSink(...);  
Sink sinkCopy = fileSink;
```

Пример: логирование

- При публичном наследовании объект класса-наследника неявно приводится к типу базового класса
- $\Rightarrow$ Такой код скопируется, хотя смысла в нём немного:

```
FileSink fileSink(...);  
Sink sinkCopy = fileSink;
```

- Скопируется только часть объекта `fileSink`, отвечающая классу `Sink`
- Вызов методов у `sinkCopy` вызовет методы класса `Sink`

Пример: логирование

- При публичном наследовании объект класса-наследника неявно приводится к типу базового класса
- $\Rightarrow$ Такой код скопируется, хотя смысла в нём немного:

```
FileSink fileSink(...);  
Sink sinkCopy = fileSink;
```

- Скопируется только часть объекта `fileSink`, отвечающая классу `Sink`
- Вызов методов у `sinkCopy` вызовет методы класса `Sink`
- Лучше всего было бы удалить (`= delete`) оператор присваивания класса `Sink`

Пример: логирование

- Исправление:

```
class Logger {  
private:  
    Sink* sinks_[100];  
    size_t size_;  
public:  
    Logger(): size_(0) {}  
  
    void addSink(Sink* sink) {  
        sinks_[size_++] = sink;  
    }  
  
    ~Logger() {  
        for (size_t i = 0; i < size_; ++i) {  
            delete sinks_[i];  
        }  
    }  
};
```

Пример: логирование

- Исправление:

```
class Logger {
private:
    Sink* sinks_[100];
    size_t size_;
public:
    Logger(): size_(0) {}

    void addSink(Sink* sink) {
        sinks_[size_++] = sink;
    }

    ~Logger() {
        for (size_t i = 0; i < size_; ++i) {
            delete sinks_[i];
        }
    }
};
```

- Как сделать ещё лучше?

Пример: логирование

- Исправление:

```
class Logger {  
private:  
    Sink* sinks_[100];  
    size_t size_;  
public:  
    Logger(): size_(0) {}  
  
    void addSink(Sink* sink) {  
        sinks_[size_++] = sink;  
    }  
  
    ~Logger() {  
        for (size_t i = 0; i < size_; ++i) {  
            delete sinks_[i];  
        }  
    }  
};
```

- Как сделать ещё лучше? Использовать умные указатели

- У реализации нового класса `sink` можно забыть реализовать метод `log()`, и тогда вызовется метод базового класса

Pure virtual

- У реализации нового класса sink можно забыть реализовать метод `log()`, и тогда вызовется метод базового класса
- Решение – *чисто виртуальный метод (pure virtual method)*:

```
class Sink {  
public:  
    ...  
    virtual void log(Level level, const char* message) = 0;  
};
```

Pure virtual

- У реализации нового класса sink можно забыть реализовать метод `log()`, и тогда вызовется метод базового класса
- Решение – *чисто виртуальный метод (pure virtual method)*:

```
class Sink {  
public:  
    ...  
    virtual void log(Level level, const char* message) = 0;  
};
```

- Класс с чисто виртуальными методами называется *абстрактным*

Pure virtual

- У реализации нового класса sink можно забыть реализовать метод `log()`, и тогда вызовется метод базового класса
- Решение – *чисто виртуальный метод (pure virtual method)*:

```
class Sink {  
public:  
    ...  
    virtual void log(Level level, const char* message) = 0;  
};
```

- Класс с чисто виртуальными методами называется *абстрактным*
- Создать экземпляр абстрактного класса **нельзя** (будет ошибка компиляции), но можно от него отнаследоваться

Pure virtual

- У реализации нового класса sink можно забыть реализовать метод `log()`, и тогда вызовется метод базового класса
- Решение – *чисто виртуальный метод (pure virtual method)*:

```
class Sink {  
public:  
    ...  
    virtual void log(Level level, const char* message) = 0;  
};
```

- Класс с чисто виртуальными методами называется *абстрактным*
- Создать экземпляр абстрактного класса **нельзя** (будет ошибка компиляции), но можно от него отнаследоваться
- Если класс-наследник не реализует часть чисто виртуальных методов базового класса, то он останется абстрактным

Static vs dynamic dispatch

- Как работают виртуальные функции внутри, на уровне бинарного кода?

Static vs dynamic dispatch

- Как работают виртуальные функции внутри, на уровне бинарного кода?
- Если у объекта вызван не виртуальный метод, он вызывается как обычная функция (*почти, см. thiscall*) – это т.н. *static dispatch*

Static vs dynamic dispatch

- Как работают виртуальные функции внутри, на уровне бинарного кода?
- Если у объекта вызван не виртуальный метод, он вызывается как обычная функция (*почти, см. thiscall*) – это т.н. *static dispatch*
- Если функция виртуальная, нужно понять, какого типа на самом деле объект, и вызвать соответствующий метод – это т.н. *dynamic dispatch*

Static vs dynamic dispatch

- Как работают виртуальные функции внутри, на уровне бинарного кода?
- Если у объекта вызван невиртуальный метод, он вызывается как обычная функция (*почти, см. thiscall*) – это т.н. *static dispatch*
- Если функция виртуальная, нужно понять, какого типа на самом деле объект, и вызвать соответствующий метод – это т.н. *dynamic dispatch*
- В C++ (и многих других ООП-языках) *dynamic dispatch* работает с помощью *таблицы виртуальных функций*

Таблица виртуальных функций

- Для каждого класса с виртуальными методами компилятор создаёт особый массив указателей на функции (один на класс)

Таблица виртуальных функций

- Для каждого класса с виртуальными методами компилятор создаёт особый массив указателей на функции (один на класс)
- Для каждого виртуального метода где-то в этом массиве лежит указатель на него

Таблица виртуальных функций

- Для каждого класса с виртуальными методами компилятор создаёт особый массив указателей на функции (один на класс)
- Для каждого виртуального метода где-то в этом массиве лежит указатель на него
- В каждый экземпляр класса компилятор добавляет указатель (недоступный программисту!) на этот массив в некое фиксированное место (обычно – в начало класса)

Таблица виртуальных функций

- Для каждого класса с виртуальными методами компилятор создаёт особый массив указателей на функции (один на класс)
- Для каждого виртуального метода где-то в этом массиве лежит указатель на него
- В каждый экземпляр класса компилятор добавляет указатель (недоступный программисту!) на этот массив в некое фиксированное место (обычно – в начало класса)
- При вызове виртуального метода сначала достаётся нужный элемент из этого массива, и потом он вызывается как функция

Таблица виртуальных функций

- Для каждого класса с виртуальными методами компилятор создаёт особый массив указателей на функции (один на класс)
- Для каждого виртуального метода где-то в этом массиве лежит указатель на него
- В каждый экземпляр класса компилятор добавляет указатель (недоступный программисту!) на этот массив в некое фиксированное место (обычно – в начало класса)
- При вызове виртуального метода сначала достаётся нужный элемент из этого массива, и потом он вызывается как функция
- $\implies$ Вызов виртуального метода немного дороже, чем вызов обычного метода

Таблица виртуальных функций

- Для каждого класса с виртуальными методами компилятор создаёт особый массив указателей на функции (один на класс)
- Для каждого виртуального метода где-то в этом массиве лежит указатель на него
- В каждый экземпляр класса компилятор добавляет указатель (недоступный программисту!) на этот массив в некое фиксированное место (обычно – в начало класса)
- При вызове виртуального метода сначала достаётся нужный элемент из этого массива, и потом он вызывается как функция
- $\implies$ Вызов виртуального метода немного дороже, чем вызов обычного метода
- $\implies$ Размер класса с виртуальными функциями на `sizeof(void*)` байт больше, чем кажется :)